Potluck Contest

By Arpa

A. actGenshinImp

- Implementation
- Dp
- Backtrack

A. actGenshinImp

- If the letters were unique, dp could be used.
- If the length was short, bruteforce could be used.
- Let's combine.
- The part that might make a cycle only consists of the substring "nshini".
- Backtracking only this part of the string, and running DP individually on prefix and suffix would works.
- Solving other cyclic shifts is similar.
- If one letter of the two duplicate letters moves to the other side, the distance becomes odd. As the grid is bipartite, such letters cannot make a cycle.
- So there are four cases to consider in total.
    - The nshini exists as a whole.
    - ini exists, n is on the other side.
    - nshin exists, i is on the other side.
    - Both n and i does not make cycles.

B. Bracket Problem Yet Again

- Segment tree

B. Bracket Problem Yet Again

- Greedy.
- To solve for k = 0:
  - First set all of the brackets closed.
  - Now for each i, flip one of closed brackets to open in range [1, 2 * i - 1].
  - For sure, pick the one with minimum A[i] - B[i].
  - Use std::set
- Once we remove cost from a position we won't make it have cost again.
- When removing cost, we may flip the position and then we need to flip another position too.
- To find the best choice for removing cost, we use a segment tree.

C. Candidate Elimination

- Bipartite Matching
- SCC

C. Candidate Elimination

- There exists a matching between cells and candidates.
- Hall theorem: There exists bipartite matching if every set of cells has enough candidates.

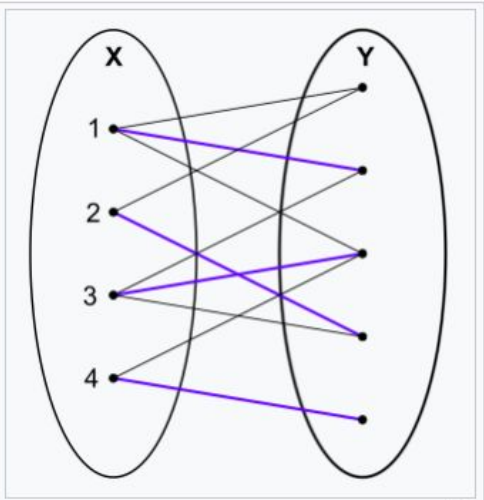
C. Candidate Elimination

Let $G = (X, Y, E)$ be a finite bipartite graph with bipartite sets X and Y and edge set E. An X-*perfect matching* (also called an X-*saturating matching*) is a matching, a set of disjoint edges, which covers every vertex in X.

For a subset W of X, let $N_G(W)$ denote the neighborhood of W in G, the set of all vertices in Y that are adjacent to at least one element of W. The marriage theorem in this formulation states that there is an X-perfect matching if and only if for every subset W of X:

$$|W| \leq |N_G(W)|.$$

In other words, every subset W of X must have sufficiently many neighbors in Y.



blue edges represent a matching

C. Candidate Elimination

- We want to find the edges that are never in matching.
- We need to find minimal tight sets: $|W| = |N_G(W)|$.
- Any edge outside tight sets never appear in matching.
- To find tight sets, make edges directed from cells to candidates, but reverse matching edges.
- Find SCC's.
- If two sides of an edge falls in different SCC components, it should be removed.

D. Depth of Cartesian Tree

- Next greater finding
- Segment Tree

D. Depth of Cartesian Tree

- #Double-Counting.
- Instead of summing up heights, sum up size of subtree.
- For each element e, find the length of the maximal interval containing e such that e is the maximum.
- This can be split into the subproblems: find the distance to the closest element to the left larger than e, and find the distance to the closest element to the right larger than e.
- Each subproblem can be solved independently.
- Let's solve the next greater element subproblem. This is a known problem offline. For online similar problem, see: https://codeforces.com/problemset/problem/526/F

D. Depth of Cartesian Tree

- To do it online, for each element i, keep j's such that their next greater element is i.
- Now queries are changing an element and getting sum, which can be solved by easy Fenwick tree or segment tree.

E. Extra Character

- Z-function
- DSU

E. Extra Character

- Simulate z-algorithm on s.
- You find linear number of constraints like $s_i = s_j$. Join them in DSU.
- Also, you find linear number of constraints like $s_i \neq s_j$.
- Now, run z-algorithm on t.
- Instead of “while (curr < n and s[curr] == s[curr - i]) ++curr” use “while (curr < n and eqf(curr, curr - i)) ++curr”. Where eqf checks if they are in same component in DSU.
- If q_i (the answer) can be increased by one (it’s not in not-equal constraints), $q_i = -1$.

F. Five Steiner

- Implementation

F. Five Steiner

- Each auxiliary point connects 3 points. Less than 3 is useless, more than 3 can be converted to two or more auxiliary points.
- There are at most 3 auxiliary points.
- We need to casework, 56 cases.
  - No auxiliary point: MST.
  - There is one auxiliary point connecting three points.
  - There are two auxiliary points, each connecting three points.
  - There are two auxiliary points, together connecting four points and each other.
  - There are three auxiliary points.
- In some cases, we need to solve convex optimization problem.

G. Get Mex Range Add Linear

- Segment Tree

G. Get Mex Range Add Linear

- Let's answer query for i.
- If no query with l = i is given before, answer is 1.
- If no query with l = i - 1 is given before (or r for that query was i - 1), answer is 2.
- ...
- We want to find the maximum j, such that there is no query on j or the maximum r for a query with l = j satisfies r < i.
- First set tail[i] = -1.
- For each query of first type: tail[l] = max(tail[l], r).
- Now find rightmost j with tail[j] < i.
- Join us in Appendix session to learn binary search on segment tree!

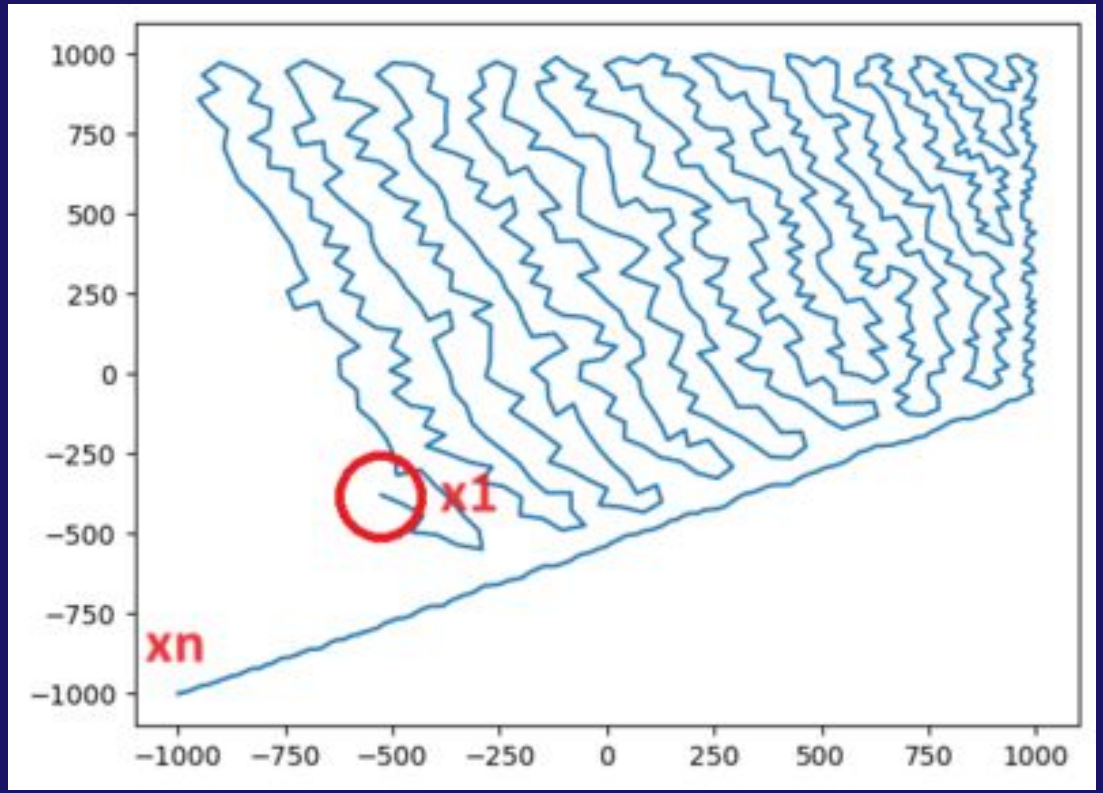
H. Hell of Optimizing Geometric Construction

- Constructive
- Implementation

H. Hell of Optimizing Geometric Construction

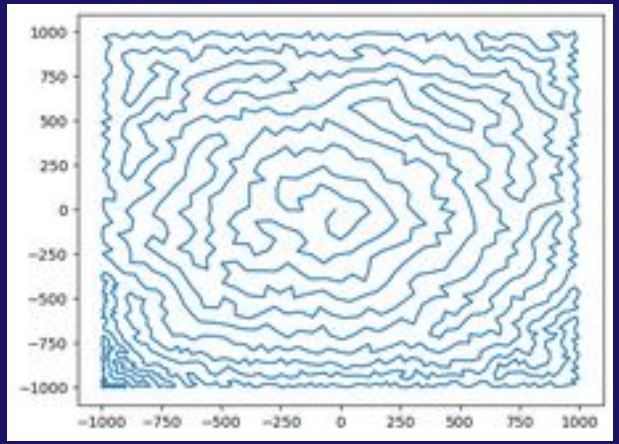
- We can set $p_i = i$ without loss of generality.
- Instead of generating from X_1 to X_n, generate from end, the shorter distances.
- Preprocess O(n) shortest vectors. Length of them will be O(sqrt(n)).
- Let's fix 0 <= delta y <= delta x for all vectors.
- Then for each vector we'll have 8 possibilities for how we want to use it.
- Use backtrack.
- Naive backtrack easily finds first 1000 points but it can't go beyond 1200.

H. Hell of Optimizing Geometric Construction



H. Hell of Optimizing Geometric Construction

- As you can see a lot of space wasted by naive backtrack.
- We should use heuristics to optimize backtrack.
- The main solution uses the heuristic to sort by decreasing distance from the origin.



I. Inequality Satisfying Subsequences

- Dp
- Binary Search

I. Inequality Satisfying Subsequences

- Sort L.
- $dp_{i, j}$: Number of subsequences ending in i, j.
- $dp_{i, j}$ = 1 + sum $dp_{k, i}$ where $L_k + L_i <= L_j$.
- Use binary search + partial sum or two pointers.

J. Jenga Tower

- Online convex hull

J. Jenga Tower

- Let's find answer for each block.
- It's easy to check if above blocks are stable.
- For below blocks, let's make two inequalities:
- We convert $l_i <= A / B$ to $0 <= A - B * l_i$.
- Similarly we get $0 <= r_i * B - A$.
- We can consider these two independently, so let's focus on $0 <= A - B * l_i$.
- For each i below the block we want to find the answer, first there was a fixed value ($A - B * l_i$) that precalculated easily. Then when we remove the current block, it's like subtracting something from A (x) and something from B (y).
- Consider two below blocks like i, j where $A_i - B_i * l_i < A_j - B_j * l_j$ and $l_i < l_j$, we can say i is less dangerous than j.
  If a x, y (from above blocks) can ruin condition for j, it'll for sure ruin condition for i.

J. Jenga Tower

- So we can remove j.
- This straightly forward directs us to use convex hull trick.
- We start from bottom, we keep dynamic convex hull and we answer queries.

K. Kaz's Party

- Guess

K. Kaz's Party

- The problem is about shuffling non-fixed points of a permutation until it's the identity permutation
- The expected value of new fixed points is 1 so maybe the expected number of rounds until reaching the identity permutation is the number of fixed points?
- Indeed that's true.
- Print N and any permutation that doesn't have any fixed point.
- The only corner case is N = 1, which takes 0 rounds because the only permutation of [1] must have a fixed point.

L. LIS on Tree

- Sack
- LIS

L. LIS on Tree

- Use sack (dsu on tree).
- Keep LDS and LIS in sack.
- Let's define LIS struct managing LIS.
- LIS struct should support two operations:
  - Adding an element to the end of array.
  - Merge info from two arrays.

L. LIS on Tree

```
template<class cmp> struct LS{
    vector<int> v;
    vector<pair<int, int>> his;
    void add(int x){
      int p = upper_bound(v.begin(), v.end(), x, cmp()) - v.begin();
      if(p == v.size()){
          his.push_back({-1, 0});
          v.push_back(x);
      }
      else{
          his.push_back({p, v[p]});
          v[p] = x;
      }
    }
    void swap(LS<cmp> &od){  od.v.swap(v);  }
```

L. LIS on Tree

```
template<class cmp> struct LS{
    void merge(LS<cmp> &od){
      if(od.v.size() > v.size())
          v.swap(od.v);
      for(int i = 0; i < od.v.size(); i++)
          if(cmp()(od.v[i], v[i]))  v[i] = od.v[i];
    }
    void undo(){
      if(his.back().first == -1)
          v.pop_back();
      else
          v[his.back().first] = his.back().second;
      his.pop_back();
    }
};
```

L. LIS on Tree

```
typedef LS<less<int> > Lis;
typedef LS<greater<int> > Lds;
```

L. LIS on Tree

- Get LIS and LDS from the big child, add v.
- Then run dfs on each small child, add vertices to current LIS and LDS. When exiting dfs of each vertex, undo the operation.
- When processing a child is done, merge it's LIS and LDS with current.

Appendix #1

Binary Search on Segment Tree

BS on Segment Tree

- Consider you have a segment tree and you want to find the smallest j (l <= j < r) with a[j] < x.
- Don't use binary search and segment tree with minimum query.
- Segment tree has built in binary search.